Advanced Data Structures - Lecture 2
Red-Black Trees

09.10.2024

Binary Trees vs Red-Black Trees

Node Structure

Binary tree fields	Red-Black tree fields

`key`
`left`,`right`: pointers to children
`parent`: pointer to parent
	`key`
	`left`,`right`: pointers to children
	`parent`: pointer to parent
	`color`: `RED` or `BLACK`

- Red-Black trees satisfy the following Binary search tree properties:
  - if x is a node with left child y then $y.\texttt{key} \leq x.\texttt{key}$
  - if x is a node with right child y then $x.\texttt{key} \leq y.\texttt{key}$

Binary Trees vs Red-Black Trees

Leaf nodes

- For binary-search trees, no memory needs to be allocated for leaf nodes. All pointers to a leaf node are assumed to be `NIL`, which is 0.
- For Red-Black trees, it is convenient to represent all leaves with a black sentinel node `NIL`:
  - The left child, right child, and parent of `NIL` are `NIL`.
  - Also, the parent of the root node of a RB tree is assumed to be `NIL`.

RB Tree properties

Red-Black tree properties (RB tree properties)

1. Every node is either black or red
2. The root is black
3. Every leaf `NIL` is black
4. The children of a red node are black
5. Every path from a node to a descendent leaf contains the same number of black nodes

Important notions and derived properties

NOTIONS:

Height $h(n)$ of a node n = number of edges in a longest path from n to a leaf.

Black height $bh(n)$ of a node n = number of black nodes (including Nil) on the path from n to a leaf, not counting n.

DERIVED PROPERTIES:

1. $bh(n) \geq h(n)/2$ for each node n.
2. The subtree rooted at node x contains $\geq 2^{bh(x)} - 1$ internal nodes.
3. A red-black tree with n internal nodes has height $\leq 2 \log_2(n+1)$.

Implementation

Binary search tree:

```
struct Node {
   int key;
   Node* left;
   Node* right;
   Node* parent;
   ...
}
```

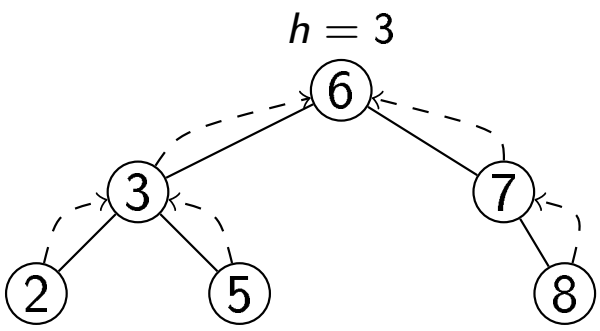
```
struct BSTree {
   Node* root;
   BSTree() { root = 0; }
   ...
}
```

Red–Black tree:

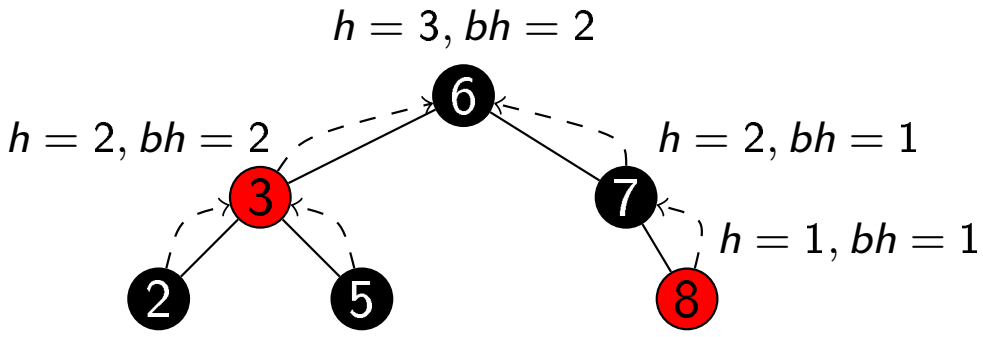
```
struct RBNode {
   int key;
   RBNode* left;
   RBNode* right;
   RBNode* parent;
   enum Color {RED, BLACK};
   Color color;
   ...
}
```

```
struct RBTree {
   RBNode* root;
   RBTree() { root = RBNode::Nil; }
   ...
}
```

Graphical representation



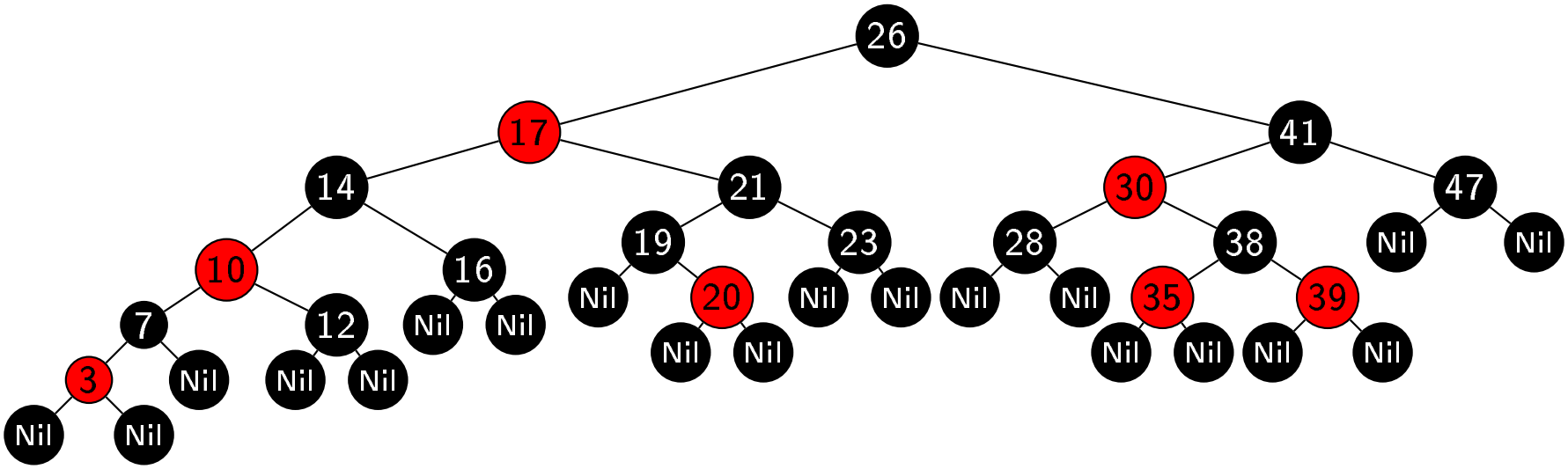
Binary search tree



Red–Black tree

- Nodes are represented by circles with the key value written inside.
  - Nodes may contains more data, not shown in the graphical representation.
  - For Red–Black trees, the circles are colored with the node color.
- Thick lines between nodes are the pointers from parent to children.
- Dashed arrows are pointers from node to parent; they are usually not drawn.

Example



Balanced Red–Black Tree

Operations on Red–Black trees

Assumption: S is an instance of a `RBTree`, which represents a red-black search tree with n nodes.

Operation	average case	worst case
S.`search`(k)	$O(\log_2 n)$	$O(\log_2 n)$
S.`minimum`()	$O(\log_2 n)$	$O(\log_2 n)$
S.`maximum`()	$O(\log_2 n)$	$O(\log_2 n)$
S.`successor`(x)	$O(\log_2 n)$	$O(\log_2 n)$
S.`predecessor`(x)	$O(\log_2 n)$	$O(\log_2 n)$

- These are query operations, implemented the same way as for binary trees.
- The modifying operations `insert` and `delete` (`del`) must be redesigned carefully, to guarantee that the newly produced tree has the red-black properties.

Insertion and deletion in a Red–Black tree

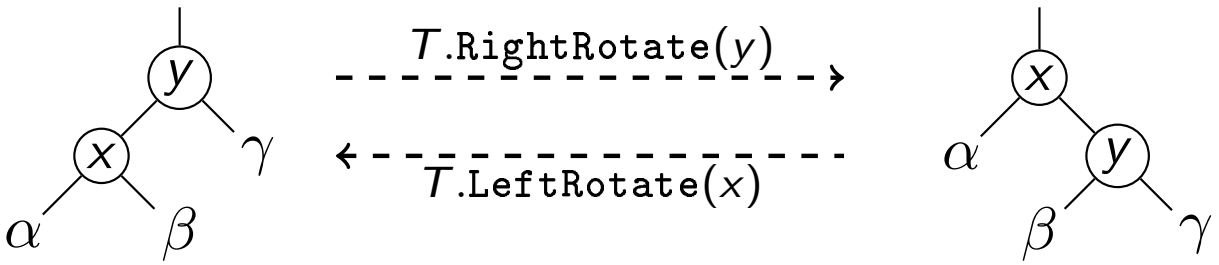
- Operations `insert` and `del` for binary search trees can be applied also on Red–Black trees with n keys.
  - ⇒ Execution time is $O(\log_2(n))$.
  - ⇒ Can violate the red-black properties of the tree:

    insert a RED node : can violate property 4;
    a BLACK node: can violate property 5.
    del a RED node: does not violate any property;
    a BLACK node: can violate properties 2, 4, 5.

- **the red-black properties must be restored:**
  - ⇒ Some nodes must change color
  - ⇒ Some pointers must be changed.

LeftRotate și RightRotate

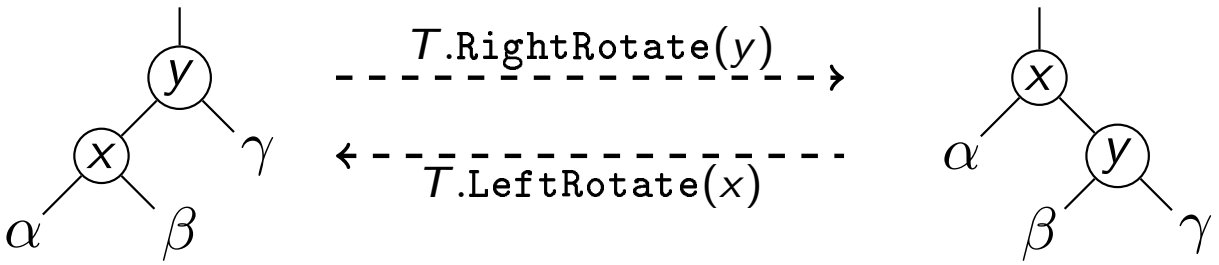
Rotation operations on Red–Black trees



- T.RightRotate(x) transforms the configuration of the two nodes on the left into the configuration on the right by changing a constant number of pointers.
- The configuration on the right can be transformed into the configuration on the left by the inverse operation T.LeftRotate(y). The two nodes may appear anywhere in a binary search tree T. The leeters α, β, și γ represent binary subtrees.

LeftRotate și RightRotate

Rotation operations on Red–Black trees



- T.`RightRotate`(x) transforms the configuration of the two nodes on the left into the configuration on the right by changing a constant number of pointers.
- The configuration on the right can be transformed into the configuration on the left by the inverse operation T.`LeftRotate`(y). The two nodes may appear anywhere in a binary search tree T. The leeters α, β, și γ represent binary subtrees.
- A rotation operation preserves the inorder ordering of keys: the keys in α precedes x-`>key`, which precedes the keys in β, which precede y-`>key`, which precedes the keys in γ.

LeftRotate și RightRotate

Assumptions: x->right $\neq$Nil.

```
LeftRotate(Node* x) // method of BSTree
y = x->right
x->right = y->left
if y->left ≠ Nil
   y->left->parent = x
y->parent = x->parent
if x->parent == Nil
   root = y
else if x == x->parent->left
   x->parent->left = y
else x->parent->right = y
y->left = x
x->parent = y
```

- Source code of RightRotate is similar.
- LeftRotate and RightRotate run in $O(1)$ time.

Insertin in Red–Black trees

Main idea

In order to insert a node x in a Red–Black tree T with n nodes, proceed as follows:

- Perform insertion of x in T, as if T were a binary search tree.
- Color x to be RED.
- Adjust the color of the modified tree, by recoloring nodes and performing rotations.
- These ideas are implemented in the tt RBInsert procedure.

Insert

RBInsert

```
RBInsert(Node* z) // method of the class RBNode
y = Nil
x = root
while x ≠ Nil
    y = x
    if z->key < x->key
        x = x->left
    else x = x->right
z->parent = y
if y == Nil
    root = z
else if z->key < y->key
    y->left = z
else y->right = z
z->left = Nil
z->right = Nil
z->color = RED
RBInsertFixup(z)
```

Insert

Analysis of RBInsert

- RBInsert ends by coloring the new node z red.
- Then it calls RBInsertFixup because we could have violated some red-black properties:
  - Properties 1, 3, and 5 are not violated.
  - Property 2 is violated if z is the root.
  - Property 4 is violated if z->parent is RED, because both z and z->parent sunt RED.

$\Rightarrow$ These violations are removed by calling RBInsertFixup.

Insert

RBInsertFixup

```
RBInsertFixup(Node* z) // method of class RBNode
while z->parent->color == Red
    if z->parent == z->parent->parent->left
        y = z->parent->parent->right
        if y->color == RED
            z->parent->color = BLACK                    Case 1
            y->color = BLACK                            Case 1
            z->parent->parent->color = RED              Case 1
            z = z->parent->parent                       Case 1
        else if z == z->parent->right
                z = z->parent                           Case 2
                LeftRotate(z)                           Case 2
            z->parent->color = BLACK                    Case 3
            z->parent->parent->color = RED              Case 3
            RightRotate(z->parent->parent)              Case 3
    else
        (same as then clause with right and left exchanged)
root->color = BLACK
```

Remark: the following loop invariants hold at the start of each **while** loop:

- z is RED
- At most one red-black violation: z is RED (property 2), or z->p are both RED

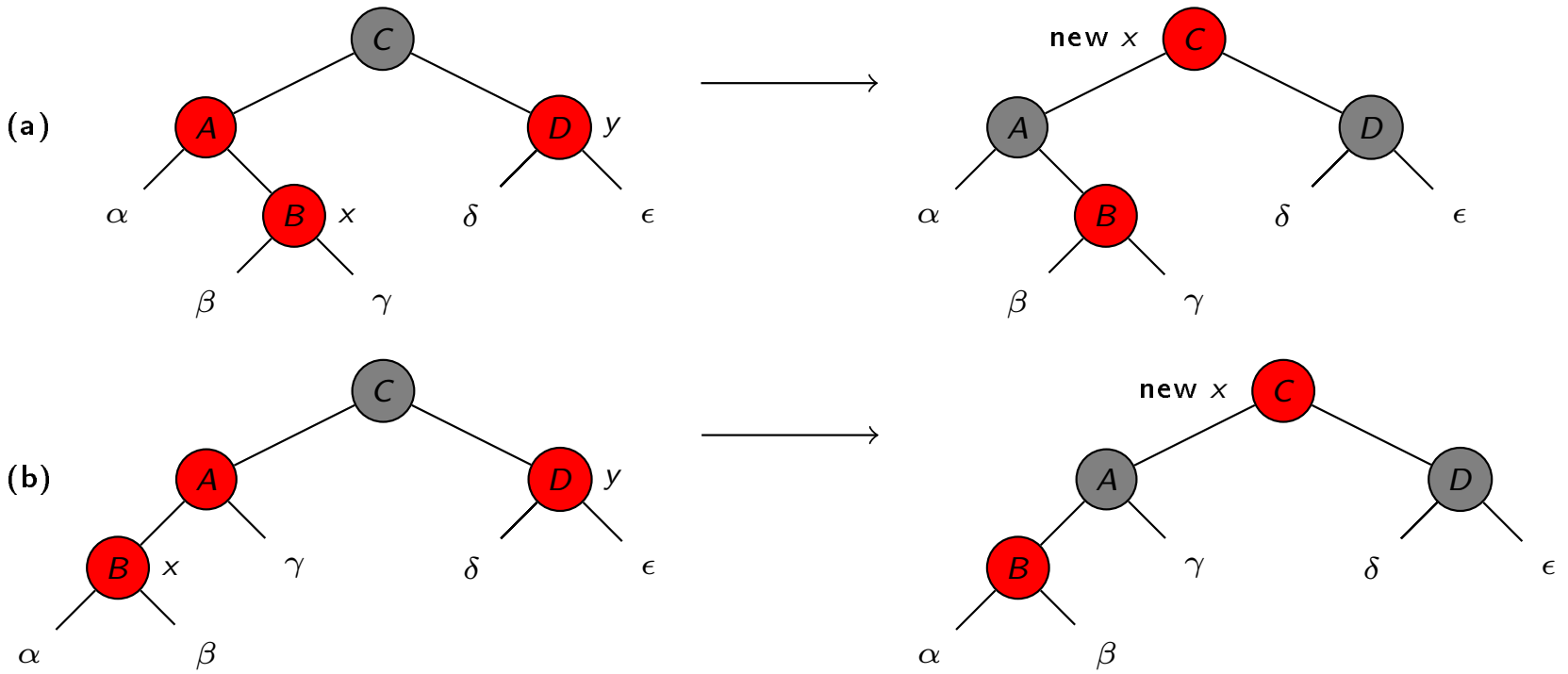
Insert

Analysis of RBInsertFixup

- The **while** loop represents 6 cases; three of them are symmetrical, depending if `z->parent` is the left or right son of `z->parent->parent`. ⇒ only three cases should be considered.
- Case 1 analyze the color of `z->parent->parent->right`. Line 3 makes y point towards `z->parent->parent->right`, and we test it on line 4. If y is red, then execute case 1, otherwise, either case 2 or case 3 are executed. For all cases, `z->parent->parent` is black (reason: `z->parent` is red), and property 3 is violated between z and `z->parent`.
- For cases 2-3, uncles color of z: y is black. The two cases can be distinguished as follows: : z is a right son, or left son of `z->parent`. Lines 10-11 represent case 2. In this situation, we do a left rotation, transforming case 3 (lines 12-14) where z is left son. Regardless on how we get into case 2, the uncle of z, y is black (reason: otherwise we would be in case 1). In this situation we change colors and do a right rotation, hence property 4 will hold. This operation leads to completion of execution because `z->parent` is black now.

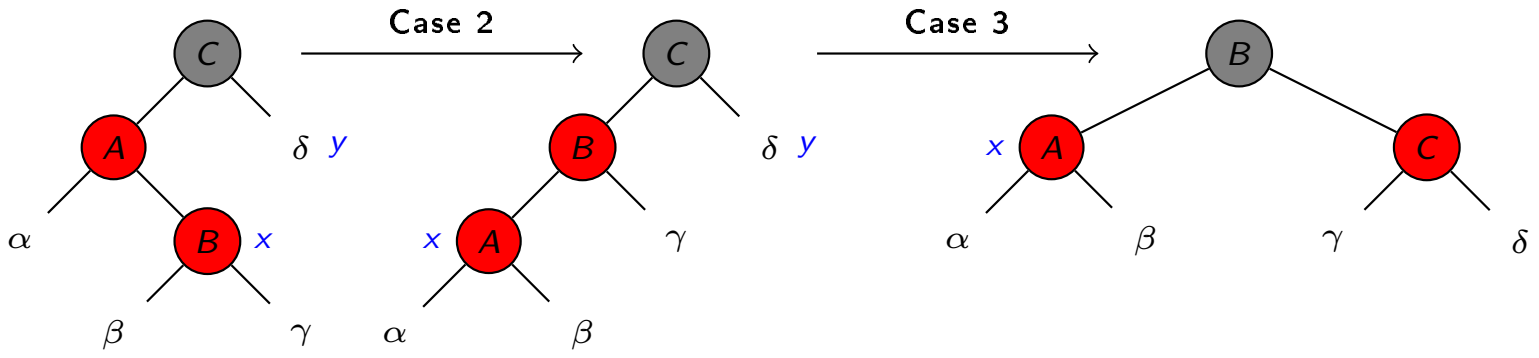
Execution time of the T.RBInsert(x) operation is $O(\log_2(n))$, where n is the number of nodes from the Red–Black tree T.

Visually: Case 1 T.RBInsert(x)



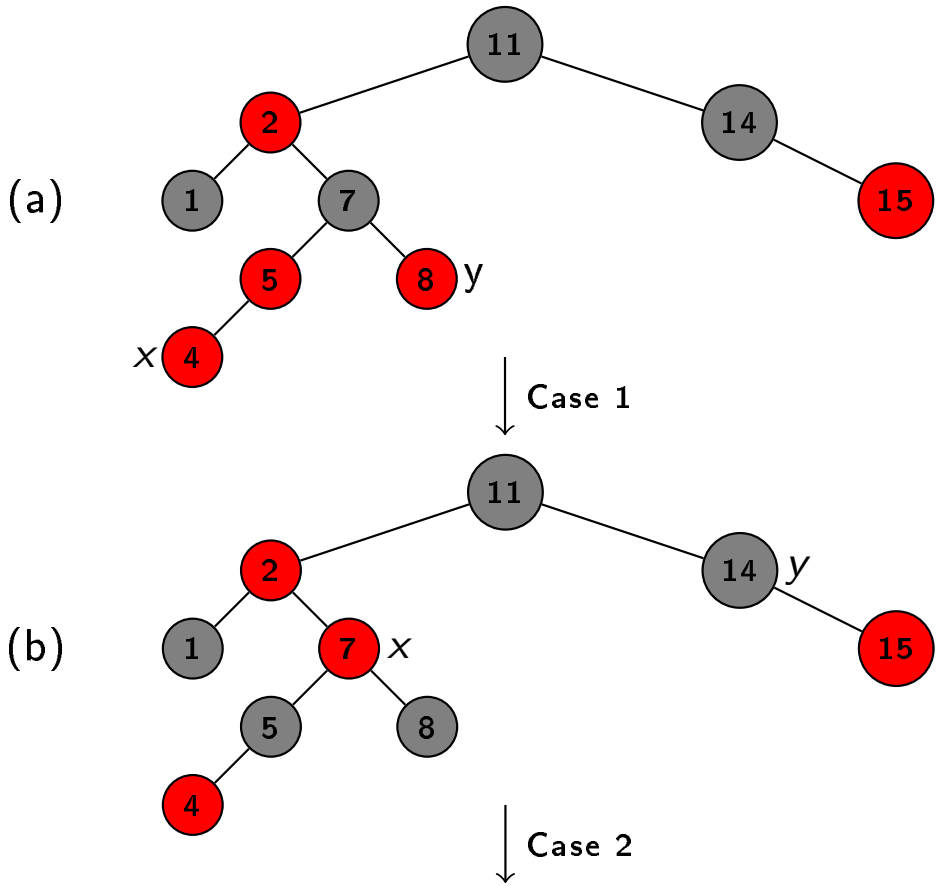
- Property 4 is violated, because both x and its parent are red. Same action will happen if (a) x is a right son or (b) x is a left son. Each of the subtrees α, β, γ, δ, and ϵ have black root and same black height. Sourcecode for case 1 will change some node colors in order to preserve orioerty 5. The **while** loop continues with the grandparent of the node x: `x->parent->parent` as being the new x. Any violation of property 4 can appear only between new x (red) and his parent, if this is also red.

Visually: Case 2 and 3 T.RBInsert(x)



- Similar to case 1, propriety 4 is violated both in case 2 and case 3 because x and its parent x->parent are red. Every subtree α, β, γ, and δ have black root and similar black height. Case 2 is converted into case 3 by a left rotation, hence we preserve property 5. Case 3 will change some colors and a right rotation, also preserving property 5. **while** loop ends because property 4 is fulfilled.

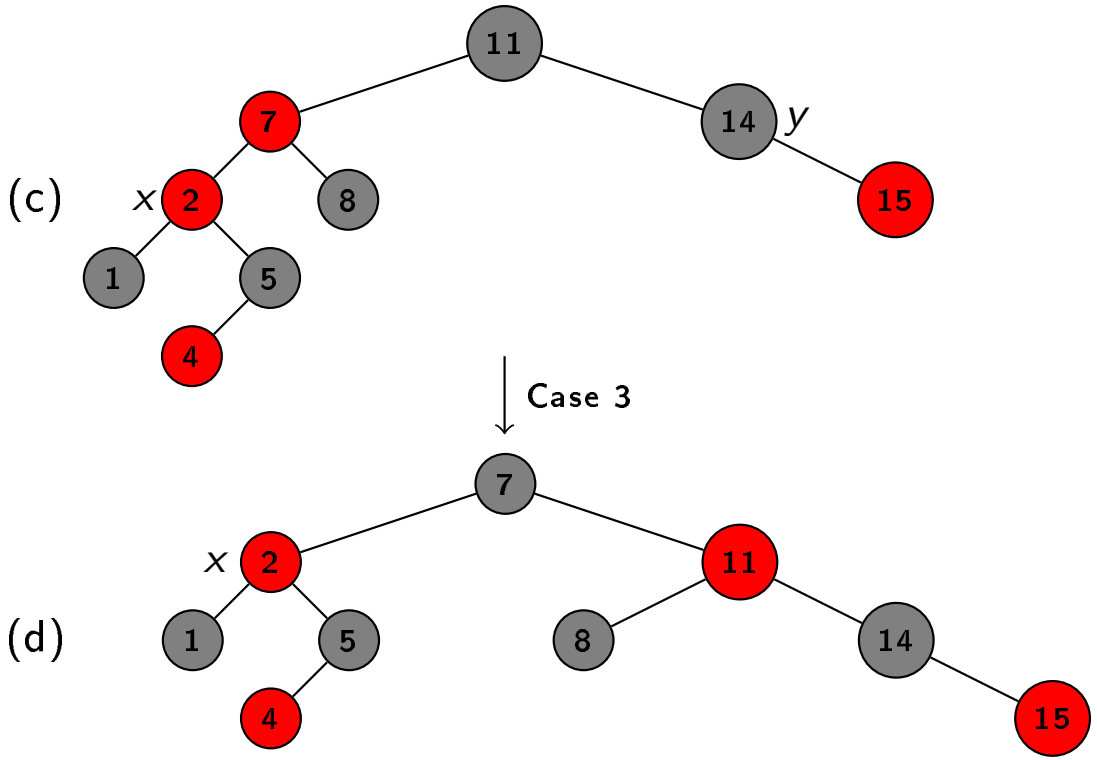
Visually: T.RBInsert(x)



(a) Node x after insertion. Both nodes x and x->parent are red breaking the red-black property. Since the uncle of x, y is red, apply case 1: nodes are re-colored and the pointer of x points upwards.

(b) x and x->parent are both red but the uncle of x, y, is black. Hence x being a right son, we apply case 2 (uses a left rotation).

Visually: T.RBInsert(x)



(c) x is a left son $\Rightarrow$ execute case 3.
Perform a right rotation, we will obtain the tree from (d), which is a correct Red–Black tree.

Deletion

- We will present a procedure T.`RBDelete`(x) that performs the deletion of node x form the RB-tree T in $O(\log_2(n))$.
- T.`RBDelete`(x) is a subtle adjustment of the deletion procedure for the binary search trees. After splicing out a node, it calls an auxiliary procedure T.`RBDeleteFixup`(x) that changes colors and performs rotations to restore the red-black properties.

Deletion

RBDelete vs. del

```
RBDelete(z)
if z->left == Nil or z->right == Nil
  y := z
else y = successor(z)
if y->left ≠ Nil
  x = y->left
else x = y->right
x->parent = y->parent
if y->parent = Nil
  root = x
else if y == y->parent->left
  y->parent->left = x
else y->parent->right = z
if y ≠ z
  z->key = y->key
  if y has other fields,
  copy in z
if y->color = BLACK
  RBDeleteFixup(x)
return y
```

```
del(z)
if z->left == Nil or z->right == Nil
  y := z
else y = successor(z)
if y->left ≠ Nil
  x = y->left
else x = y->right
if x ≠ Nil
   x->parent = y->parent
if y->parent = Nil
  root = x
else if y == y->parent->left
  y->parent->left = x
else y->parent->right = z
if y ≠ z
  z->key = y->key
  if y has other fields,
  copy in z
return y
```

Deletion

Analysis of RBDelete

- The check $x \neq$ Nil from line 7 is gone; instead we do the assignment x->parent $=$ y->parent from line 7 in RBDelete:
  - If x is Nil, parent pointer will point to nodes y parent.
- RBDeleteFixup(x) is called, line 16-17, if y is black.
  - If y is red, the red-black properties are not affected when y is deleted/moved because none of the black heights changed; also no red nodes will be adjacent.
  - The argument x sent in RBDeleteFixup is the only son of y before y is being removed/moved if y had a non-Nil son or otherwise the sentinel node Nil. In the latter case line 7 guarantees x->parent will become previous parent of y, if x is a regular node or sentinel node Nil.

Deletion

Analisys of RBDelete

- If node y that will be deleted from RBDelete is black, then it will pose the following challenge: all the paths that contained y have one black node less $\Rightarrow$ Violation on property 5, for all the parents of y from the tree.

$\Rightarrow$ Fixing it: adding 1 to the number of black nodes from each path that contain x we obtain:

  - Satisfy property 5;
  - When deleting a black node y, "add" black characteristics to its sons.
  - Propriety 1 can be violated, when x can be "twice black" (doubly black) or "red and black".

- Calling the method RBDeleteFixup(x) is done in order to re-establish property 1.

RBDeleteFixup

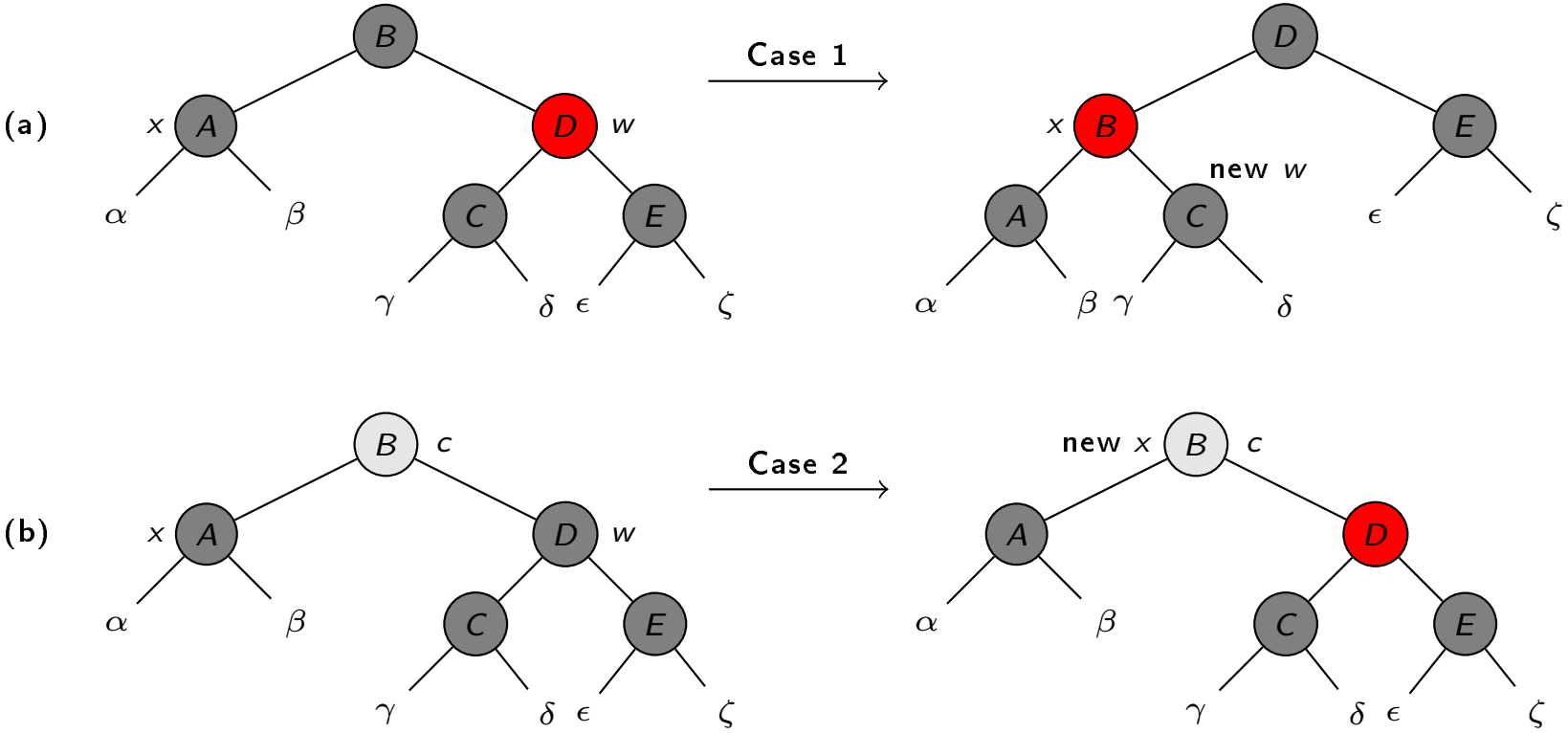
```
RBDeleteFixup(x)
while x ≠ root and x->color == BLACK
    if x == x->parent->left
      w = x->parent->right
      if w->color = RED
        w->color = BLACK                                        Case 1
        x->parent->color = RED                                  Case 1
        LeftRotate(x->parent)                                   Case 1
        w = x->parent->right                                    Case 1
      if w->left->color == BLACK and w->right->color == BLACK
        w->color = RED                                          Case 2
        x = x->parent                                           Case 2
      else if w->right->color == BLACK
            w->left->color = BLACK                              Case 3
            w->color = RED                                      Case 3
            RightRotate(w)                                      Case 3
            w = x->parent->right                                Case 3
        w->color = x->parent->color                             Case 4
        x->parent->color = BLACK                                Case 4
        w->right->color = BLACK                                 Case 4
        LeftRotate(x->parent)                                   Case 4
        x = root                                                Case 4
    else (similar to the clause then with right and left inter-changed)
x->color = BLACK
```

Deletion

Analysis `RBDeleteFixup`

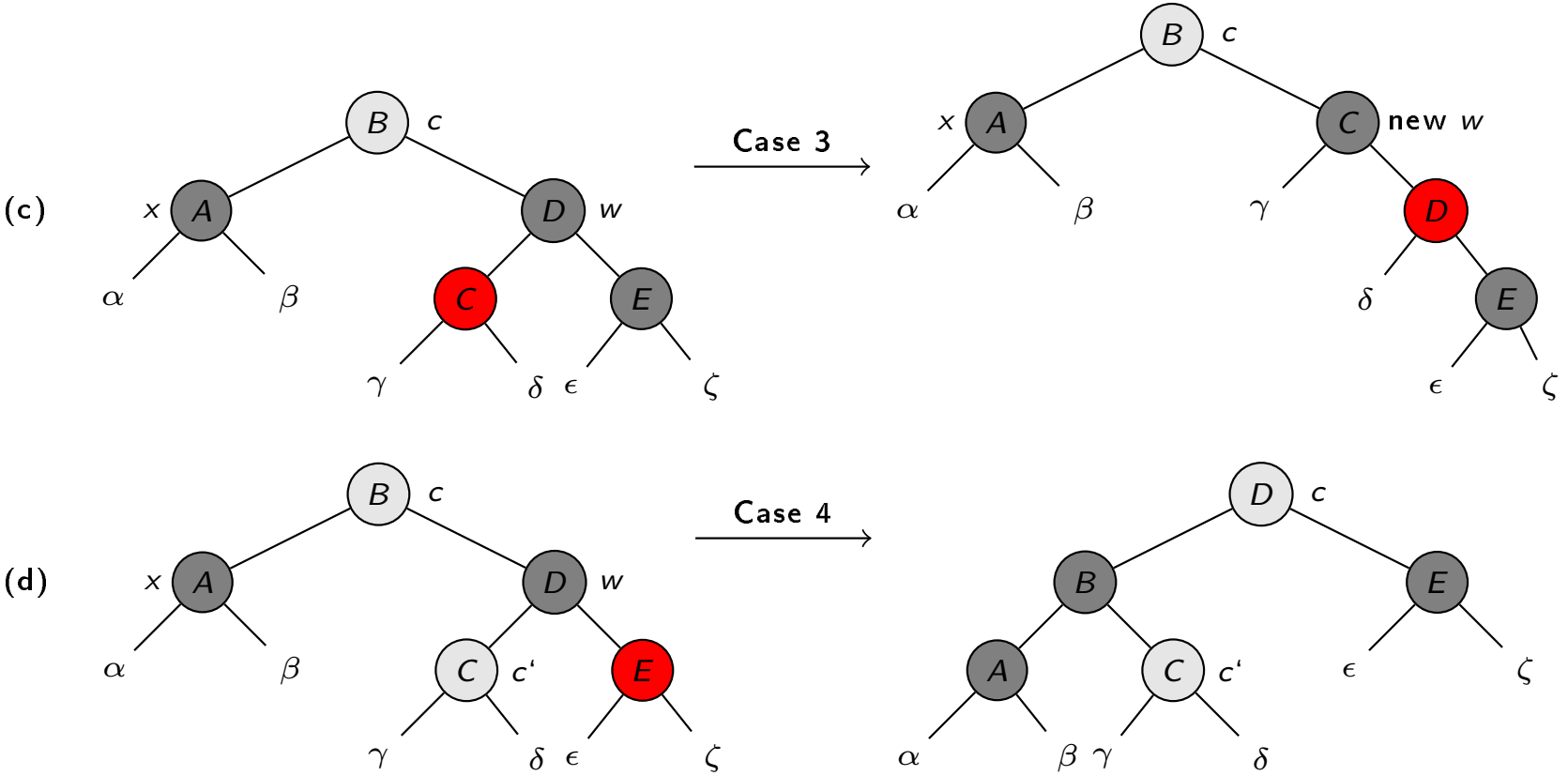
- Lines 1-22 are in charge of moving the extra black node upwards in the tree until:
  1. x points to a red node $\Rightarrow$ color the node black (line 23)
  2. x points to the root $\Rightarrow$ extra black node can be simply "removed"
  3. Rotations and re-colorings can be applied.
- In lines 1-22, x always points towards a black node that is not the root, but that has an extra black node where w is set to point towards the brother of x. The **while** loop presents 8 distinct cases. Four of them are symmetric with the other ones, depending if x is left son or right son of x`->parent`.

Visually: Case 1 și 2 T.RBDeleteFixup(x)



- *Dark nodes* are black, *light grey nodes* can be red or black.
- Greek letters represent sub-trees.
- The node pointed by x has an extra black son.

Visually: Case 3 and 4 T.RBDeleteFixup(x)



- *Dark nodes* are black, *light grey nodes* can be red or black.
- Greek letters represent sub-trees.
- The node pointed by x has an extra black son.

RBDeleteFixup:Implementation analysis

The four cases inside the **while** loop

The only case that can cause the loop to repeat is **case 2**.

(a) Case 1 is transformed into cases 2, 3, or 4 by exchanging colors of nodes B and D and performing a left rotation.

(b) In Case 2, the extra black represented by the pointer x is moved up the tree by coloring node D red and setting x to point to B. If we enter case 2 through case 1, the **while** loop terminates since the color c is red.

(c) Case 3 is transformed to case 4 by exchanging the colors of nodes C and D and performing a right rotation.

(d) Case 4, the extra black represented by x can be removed by changing some colors and performing a left rotation.

RBDelete

Time complexity analysis

- The height of the RB-tree is $O(\log_2(n))$. Hence:
  - $\Rightarrow$ the total cost of the procedure without the call to `RBDeleteFixupcostul` is $O(\log_2(n))$.
- With the call to the `RBDeleteFixup`, cases 1, 3, 4 each terminates after a constant number of color changes and ≤ 3 rotations. Case 2 is the only case which the **while** loop can be repeated, and then the pointer x moves upward at most $O(\log_2(n))$ times, and no rotations are performed. Thus:
  - $\Rightarrow$ `RBDeleteFixup`(x) takes $O(\log_2(n))$ time and performs at most 3 rotations.

$\Rightarrow$ Total execution time for `RBDelete`(x) is also $O(\log_2(n))$.

Reference

- Chapter 13 (Red-Black Trees) from the book
  - ⇒ T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein: *Introduction to Algorithms*. 3rd Ed. MIT Press. 2009.

- Next lecture:
  - ⇒ B Trees – Chapter 18 (B-Trees)